Homework: running the full dataset and all the steps

In the workshop we processed a handful of images with a pretrained model to see how each step behaves.
This guide is for running the full *Listeria* training set on your own computer, in your own time.
Feature extraction and model training on the full set will take a while.
You will run the same pipeline you have already used, just at full scale.

Before you start

- Download the full training set from Zenodo: https://zenodo.org/records/20377151/files/17_11_2023_3D_z_21.zip?download=1
- Unzip it and keep all images in a single folder. It is a large file (~50 GB) — make sure you have enough free disk space before you begin.

Prepare your laptop for a long run. A full run writes a large feature table and must not be interrupted. Plug in the power, disable sleep, and postpone system updates for the duration. Start the MicroICS (Look document: **Installation guide**).

Generate features

This is the step to leave running overnight, or across two days, depending on how many parallel workers your machine supports. The software measures around 2,700 descriptors for every image, and on the full set that is a substantial amount of computation.

Do: Run feature extraction in labelled mode on the full image folder. How to do this is described in the "**Hands on exercise**" - Generate features.

Outputs:
When processing is complete, datafile.tsv contains one row per image. Open it in Excel and check the first column: every image should be listed by name. Any missing images indicate a naming or format problem with those specific files.

Train a model

Once datafile.tsv is complete, run the training.

Do: Run training. How to do this is described in the "**Hands-on exercise**" – Train a model.
Use all learners to compare algorithms, or a single learner if you already know which one you want. For this dataset, random forest is the recommended choice.

Running all learners takes a while — comparable to feature extraction — so it is worth doing once, to see which method performs best, and thereafter running only the best-performing model.

Outputs:
This step produces the trained model and the performance results together.

The outputs to examine first are those from the workshop: classification_all, rankings_label, rankings_date, and ablation_ranking_all. Full descriptions of every output file are given below.

File	What it tells you
classification_all	Report on the performance of each classifier (provided as a table and plot in the visualisation folder). Use this to select the optimal classifier.
classification_no_counts	The same as classification_all, excluding count-based features. The difference shows how much performance depends on count-related features.
ablation_ranking_all	Report on how many features are needed for optimal performance, shown as a plot in the visualisation folder. Provided only for the random forest.
rankings_label	Ranking of structural features that best separate your biological groups, which you can use to understand what drove the model decisions. Provided only for random forest.
rankings_date	Similar to rankings_label, but tracks what is important for predicting the date and refers to the batch effect. Comparing this with rankings_label is the batch-effect check. Provided only for random forest.
Top_feature_correlation_clustered (visualisations)	Shows how strongly the top features correlate with one another and helps to identify clusters of features that correlate and may thus point to redundancy. Each cell gives the correlation between two features, so bright blocks along the diagonal reveal groups of features that are essentially measuring the same thing.
Confusion_matrix (visualisations)	For each model, a confusion matrix is provided for both all-features and no-counts features. The matrix shows how often predictions are correct and, when they are wrong, which classes are mistaken for which. Rows represent the true class and columns the predicted class; the diagonal shows correct predictions, and off-diagonal cells show errors. Classes that are repeatedly confused are structurally similar within the model, which indicates which mistakes are to be expected and helps you interpret low-confidence predictions in the right context.

Inference

Inference is run in two steps: first calculate features from the new images, then run the model on them.

Inference images (Zenodo):
https://zenodo.org/records/20390961/files/E8_images_for_prediction.zip?download=1

Do:
Run (1) “generate features” on the downloaded inference images in unlabelled mode. How to do this is described in the “**Hands-on exercise**” – Inference part 1.

Run (2) inference on the resulting features. How to do this is described in the "**Hands-on exercise**" – Inference part 2.

Outputs:
The outputs to examine first are those from the workshop: inference_summary, rf_predictions, rf_probabilities. Full descriptions of every output file are given below.

File	What it tells you
inference_summary	This report provides an overview of the run – model used, number of classes, predictions made.
rf_predictions	Report the predicted class for each image, chosen by majority vote.
rf_probabilities	Report the confidence behind each prediction. Values near 1 indicate a clear match; low values indicate that the image resembles more than one class or a structure not in the training set.
rf_features	The feature values calculated for the new images may be especially useful for checking that any external features you added were read correctly.
rf_feature_importance (explanations)	Report on feature importance for the overall dataset (all classes combined). Each feature gets a value and is ranked based on this value, indicating how important it was for the entire dataset.
rf_feature_importance_ class (explanations)	A bar chart for each class showing which features most influenced the prediction for that class. Only the top features are shown.
rf_shap_class (explanations)	Per-image SHAP values are provided in the table, which forms the basis for the summary plots described below.
Rf_summary_plot (explanations)	A beeswarm plot is provided for the entire dataset, showing all classes together. Each dot represents an image; its colour indicates the feature value, and its position shows the direction of the predictions.
rf_shap_plot (explanations)	Similar to the beeswarm plot above, but provided separately for each class.